

李乐岩

leyanpower@gmail.com | +86 18805506898 | GitHub: github.com/leyancode | 个人主页: leyan-li.com

爱丁堡大学 University of Edinburgh 英国 | 2025.09 - 2026.12 高性能计算与数据科学 硕士 课程与毕业论文 2026.08 完成 | 可入职时间 2026.09.01 | 学位证书 2026.12 获得

利物浦大学 University of Liverpool 英国 | 2023.09 - 2025.06 计算机科学 本科, 西交利物浦大学 2+2 项目, 一等荣誉学位

关联课业: 消息传递并行编程 (MPI)、线程并行编程 (OpenMP)、HPC 体系结构、性能编程与优化、GPU / 加速器编程、并行设计模式、高性能数据分析、机器学习

技术能力

编程语言: C / C++、Python、Bash、SQL

并行与性能工程: MPI、OpenMP、hybrid MPI+OpenMP、PETSc、Slurm、numactl / hwloc、Linaro MAP; strong / weak scaling、parallel efficiency 与饱和点判定, halo exchange、global reduction (allreduce) 等集合通信开销, roofline / 访存墙估算, NUMA 与线程绑定

深度学习与大模型: PyTorch、NumPy、pandas; LLM function calling / tool schema 设计、离线评测集构建与消融实验。GPU 与推理服务为课程与自学层面: CUDA 编程模型、NCCL 集合通信、数据 / 张量并行、batching 与吞吐-时延取舍、KV cache 与量化对每 token 访存量的影响

工程与工具链: Linux、Git、pytest、GitHub Actions CI、FastAPI + Pydantic、SQLite / MySQL, 了解 Docker; 把实验做成可复现管线: 参数化提交脚本 → 基于内容的日志审计 → pandas 聚合 → 回归测试

项目经历

ARCHER2 上 4,096 核规模的并行布局选型与性能归因——通信暴露与线程空转的权衡 | C · PETSc · MPI · OpenMP · Slurm · Linaro MAP · Python / pandas · GitHub Actions github.com/leyancode/hpc_benchmark_archer2

项目描述: 在英国国家超算 ARCHER2 (双路 AMD EPYC 7742、128 核/节点, Slingshot 互连) 上研究一个受约束的配置问题: 总核数固定时, 如何在 MPI rank 数与每 rank 的 OpenMP 线程数之间切分。负载为 2D/3D stencil + CG+GAMG, 5M-165M unknowns, 四种满节点布局 (128×1 / 64×2 / 32×4 / 16×8)。

- 布局选型:** 2D weak scaling 上 64×2 比 flat MPI 高 13.1% (165M unknowns / 32 节点、794.8M equations/s), fixed-20M strong scaling 达 29.83× 加速、93.2% 累积效率; 3D 上最佳布局改为 32×4 (最大增益 33.2%) 与 16×8 (比 flat MPI 快 3.81×) ——最佳并行度随维度、规模与每核工作量改变, 不存在单一「最优线程数」; 这与固定 batch 继续扩机器时的收益递减是同一类问题。
- 机理归因:** Linaro MAP 把布局排序解释为两条曲线的交点——threads/rank 上升时 solve window 内 MPI 占比 46.7%→6.2%、OpenMP 等待 48.3%→76.5%, 即「通信暴露」与「线程空转」的权衡; 排除 threaded BLAS 的解释 (LibSci ≤ 2.1% core-time), 并因插桩扰动运行时间而只读占比排序、不与未插桩 timing 混用。
- 可信度门禁:** 审计出 303 份日志中有 302 份的实际 MPI process 数与文件名不符, 据此废弃整阶段性能曲线, 正式结论只用 408 个通过内容审计的 run——把「从日志内容而非文件名解析 process / thread 数」写成分析入口硬校验; 提交脚本参数化生成全部正式配置, pytest 锁死头条数值, GitHub Actions 用 fake Slurm 跑通提交路径。

面向该 benchmark 数据的 Agent 式性能分析系统 | Python · SQLite · FastAPI · Pydantic · OpenAI SDK (function calling) · pytest github.com/leyancode/hpc-benchmark-agent

项目描述: 把上一项目的 PETSc benchmark 数据结构化为 SQLite, 性能分析任务封装为确定性工具: 让 LLM 只负责「选哪个工具、参数是什么」, 数值一律由确定性代码算, 从而让 Agent 的正确性变成可评测量。

- 可评测的路由:** 中文规则 router 作为可验证 baseline, 与 LLM function calling router 输出严格同构 (intent + 参数), 共用同一套自建 30 问五类中文评测集打分——规则 83.3%、LLM 100%; 消融显示同一模型去掉 system prompt 只有 63.3%, 即准确率主要由工具说明书与策略 prompt 决定, 而非模型本身。
- 防御式解析与工程化:** 未调用工具、幻觉工具名、arguments 非合法 JSON 三类异常统一收敛为显式 unknown, 绝不猜参数以避免「静默算错」; trace_id 贯穿的 JSON Lines 事件流可完整回放一次执行, FastAPI 的 POST /ask 把非法 router 拦为 422、缺 key 映射 503; 229 项自动化测试全量离线可跑、不需要 API key。

多智能体 Actor-Critic 方法中的价值函数分解研究 (本科毕业设计) | Python · PyTorch · PyMARL · MAPPO · VDN · QMIX · CTDE · PPO · PettingZoo / MPE2 github.com/leyancode/marl-value-factorisation

项目描述: 在 MAPPO 的 critic 上分别接入 VDN 加性分解与 QMIX 单调 mixing 做单变量对照——三个变体共用同一套 runner / buffer / shared RNN actor 与全部 PPO 超参, 差异只在 critic 分支。

- 反直觉结论与根因诊断:** 表达能力更强的 critic 没有赢——稀疏奖励的 Predator-Prey (300k steps) 上最简单的加性分解唯一学到正回报 (≈ 150), 单体 critic 始终为负, QMIX 的 critic loss 起手即 5.4×10^4 、200k 步后二次发散。根因是 MAPPO-QMIX 的 TD 目标沿用 value-based 的 $\max_a Q_i$ (贪婪动作) 而 actor 是 on-policy, 相减得到的 td_error 并非合法 advantage 而是系统性偏乐观的量; 非单调 mixer 的持续梯度爆炸作为负结果如实写进论文。

论文发表: Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis. International Conference on Engineering Management, Information Technology and Intelligence, EMITI 2024.